GETTING STARTED GUIDE

Beads
REPL Loop

A minimal setup for running Claude Code in a read-eval-print loop using Beads as persistent task state. Agents do the heavy lifting — you steer.

---

i What You Need

Before starting, make sure you have **Claude Code** installed and a project you want to work on. That's it. The agent will handle the rest of setup.

> **Agent-first approach:** This guide is written so you paste one prompt and the agent installs, initialises, and seeds Beads on your behalf. You shouldn't need to run commands manually.

1 Install & Initialise

Open Claude Code in your project directory and paste this prompt. The agent will install Beads, initialise it, and wire up the Claude Code hooks so every future session auto-loads context.

PROMPT TEMPLATE

Please set up Beads in this project for use with Claude Code.

Specifically:
1. Install the `bd` CLI if it isn't already present (`brew install beads` or the curl installer)
2. Run `bd init --quiet` in this directory

3. Run `bd setup claude` to install the SessionStart and PreCompact hooks
4. Confirm everything is working by running `bd doctor`

Show me the output of each step and let me know when it's done.

WHAT THE AGENT RUNS — AGENT

```
# Install (macOS)
brew install beads

# OR via curl on Linux
curl -fsSL https://raw.githubusercontent.com/steveyegge/beads/main/scripts/instal

# Initialise in your project (non-interactive)
bd init --quiet

# Wire up Claude Code session hooks
bd setup claude

# Verify everything is healthy
bd doctor
```

What the hooks do: `bd setup claude` installs a SessionStart hook that runs `bd prime` at the beginning of every Claude Code session, injecting ~1-2k tokens of current task context automatically. It also installs a PreCompact hook so context is refreshed on compaction events.

---

2 Seed Your Task Graph

Tell the agent what you want to build or fix. It will decompose the work into a Beads epic with individual tasks, set priorities, and wire up dependencies. You review and approve before any code is written.

PROMPT TEMPLATE

I want to *[describe the feature / refactor / goal]*.

Please:

1. Think through all the work required and break it into discrete tasks (each completable in a single focused session)
2. Create a Beads epic for the overall goal
3. Create individual tasks under it with accurate dependencies
4. Show me the full task graph with `bd epic status <epic-id> --json` when done

Don't write any code yet — just plan and file the tasks.

AGENT CREATES THE GRAPH AGENT

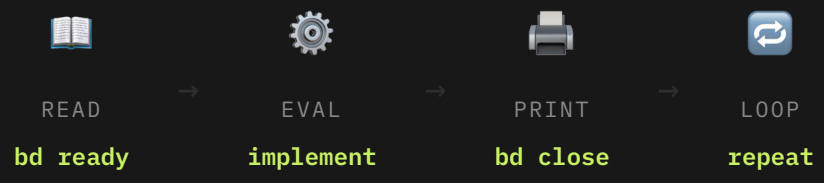
```
# Create the epic
bd create "Refactor auth module" -t epic -p 1 --json

# Create child tasks
bd create "Extract token validation" -t task -p 1 --json
bd create "Add refresh token support" -t task -p 1 --json
bd create "Write integration tests"   -t task -p 2 --json

# Wire dependencies (test blocks on the others)
bd dep add bd-c3d4 bd-a1b2
bd dep add bd-c3d4 bd-b2c3

# Review the graph
bd epic status bd-epic-id --json
```

Keep tasks small: Each task should be completable in a single focused Claude Code session. A good rule of thumb — if it takes longer than ~2 minutes of agent work, split it.

---

3 Run the REPL Loop

This is the core pattern. Each iteration is a discrete **READ → EVAL → PRINT → LOOP** cycle. The agent finds the next unblocked task, implements it, records the result, and returns to the top.



Paste this prompt to start the loop. The agent keeps going until `bd ready` returns empty:

PROMPT TEMPLATE

Start the task loop:

1. Run `bd ready --json --limit=1`
2. If empty, we're done — tell me and stop
3. Claim the task with `bd update <id> --claim`
4. Implement it
5. If you discover new required work, file it with `bd create` before closing
6. Close the task: `bd close <id> --reason "brief summary"`
7. Go back to step 1

ONE ITERATION OF THE LOOP — LOOP

```
# READ — what's unblocked?
bd ready --json --limit=1

# EVAL — claim and implement
bd update bd-a1b2 --claim
# ... agent writes code ...

# PRINT — record the result
bd close bd-a1b2 --reason "Extracted token validation to auth/tokens.go" --json

# LOOP — back to READ automatically
```

Why `--claim` matters: It atomically sets the assignee and marks the task `in_progress` in a single operation, preventing two parallel agents from picking up the same task.

4 End a Session Cleanly

When you want to stop for now, ask the agent to "land the plane." This ensures the Beads state and all code changes are pushed so the next session can resume without confusion.

PROMPT TEMPLATE

Please land the plane before we stop:

1. File any remaining work you haven't tracked yet as Beads tasks
2. Close any tasks you've completed but not closed
3. Run `git pull --rebase && git push`
4. Verify with `git status` — should be clean and up to date
5. Run `bd ready --json` so I can see what's queued for next time

LANDING SEQUENCE AGENT

```
# File any work discovered but not yet tracked
bd create "Add rate-limit tests" -t task -p 2 --json

# Close completed tasks
bd close bd-a1b2 bd-b2c3 --reason "Completed" --json

# Push everything — code AND beads state
git pull --rebase
git push

# Verify clean state
git status

# Preview next session's work
bd ready --json
```

> **Don't skip the push.** Beads state lives in git-tracked files. An unpushed session means the next session starts blind — or worse, creates conflicts.

5 Resuming Later

Because the SessionStart hook runs `bd prime` automatically, the next Claude Code session wakes up already oriented. You just need to trigger the loop again.

PROMPT TEMPLATE

> Resume where we left off. Run `bd ready --json`, pick up the top task, and continue the loop until done or I say stop.

The agent will check the task graph, find the next unblocked item, and continue — no re-explanation needed. This is the payoff: **state survived in Beads, not in the context window.**

~ Quick Reference

COMMANDS YOU'LL USE MOST — HUMAN

```
# See what's unblocked and ready
bd ready --json

# Inspect a specific task
bd show <id>

# See full epic progress
bd epic status <epic-id>

# Check for anything broken
bd doctor

# See what's blocked and why
bd blocked

# Add a task mid-loop (the agent does this, but you can too)
bd create "Fix edge case in token refresh" -t task -p 1 --json
```

> **Pro tip — use `--json` everywhere.** The agent parses JSON output much more reliably than human-readable text. Adding `--json` to any command makes the loop more robust.